

Let's design a Database

Initial Requirements :

- 1) Fast Reads
- 2) Fast Writes (More Important)
- 3) Durability
- 4) Unstructured Data
- 5) Handle a few PB of data.

Let's understand how RDBMS works first

- 1) In RDBMS we have pre-defined schema.
- 2) For every row we know the exact size & data type of columns.

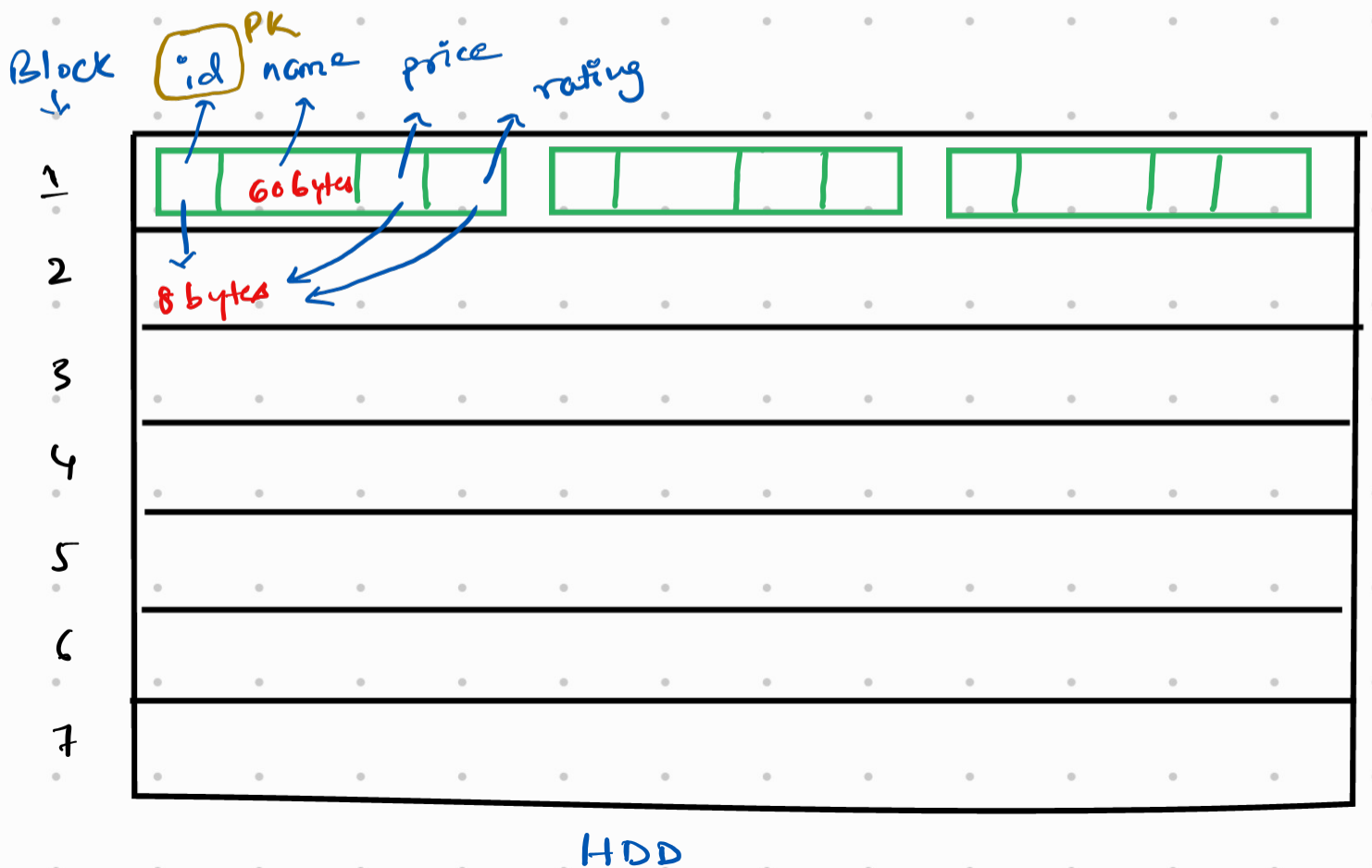
Assume a product table :-

PK →	id → 8 bytes	}	Total 84 bytes
	name → 60 bytes		
	price → 8 bytes		
	rating → 8 bytes		

- 3) Since RDBMS already knows the structure of data, so, it can perform lots of storage optimizations.

Now how will be this structured data stored on hard disk?

- RDBMS will try to store as many rows as it can in a block of disk, to optimise block usage. This is possible cuz we know beforehand the size of one row in relational DBs, and cuz of that it can make precise storage arrangements & hence reduce internal fragmentation.
- Since rows are stored in sequential manner, and so reads become fast.



Q. Now, what will happen during the reads?

→ To read the next row, we can simply apply an offset.

→ Assuming data is ordered by id, because of fixed schema and defined size, the DB knows exactly where the next record is.

How?

→ From starting byte of a row, we can jump 84 bytes ahead and reach the next row.

→ Even for updates on a specific col of a row, we just need to go to that specific location/byte by applying offset & update. This is possible cuz we already know the structure of the row. And also, updates won't change size of data.

How we improve reads on a RDBMS?

- Indexing. This will allow us to reach a row faster.
- Caching.
- Partitioning. Let's suppose we have 10 million rows in our DB. In case we wanna search for a row, we will have to perform search on all 10 million rows. If we partition our DB to 10 DB instances, and by some mechanism we know which DB contains which range of data, we can reduce our search space by 10 times, hence improving reads.

Q Can we try to think for a Key-value based no-sql how data orientation on HDD will look?

In java :-

Map < string, object >

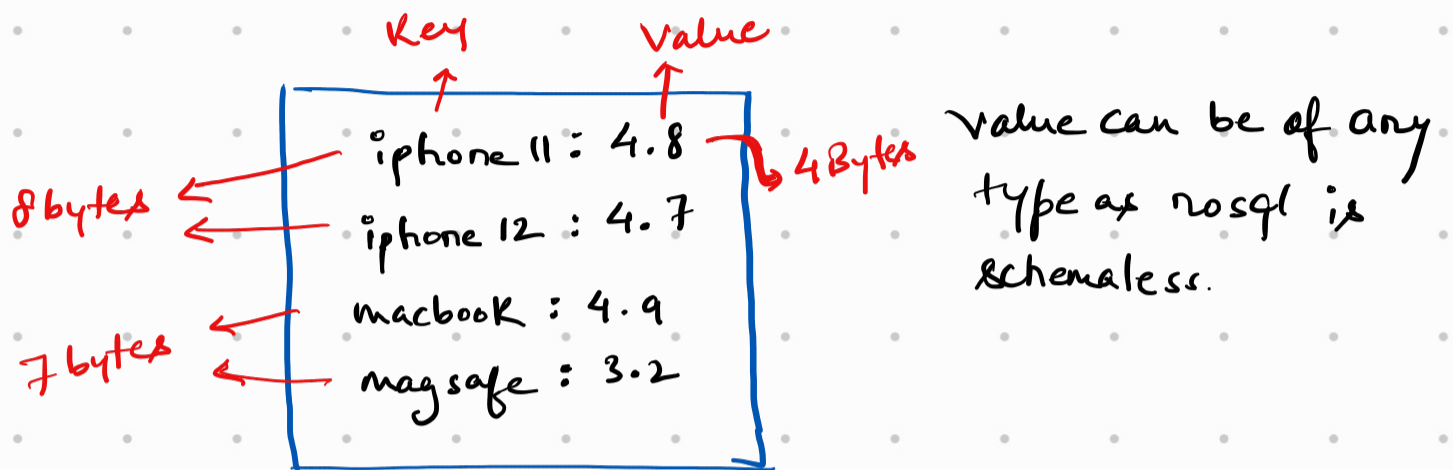
In JS :

{
Key : ...

}

⇒ These data structures are one step closer to nosql.

If, we have to store a Key-value data structure and we follow the same architecture as that of RDBMS, how things will look like?



i	p	h	o	n	e	1	1	xx	4
.	8		i	p	h	o	n	e	1
2	xx	4	.	7		m	a	c	b
o	o	k	xx	4	.	9		m	a
g	s	a	f	e	xx	3	.	2	

HDD Representation. 1 square = 1 Byte

→ yellow range shows 4 Bytes consumed

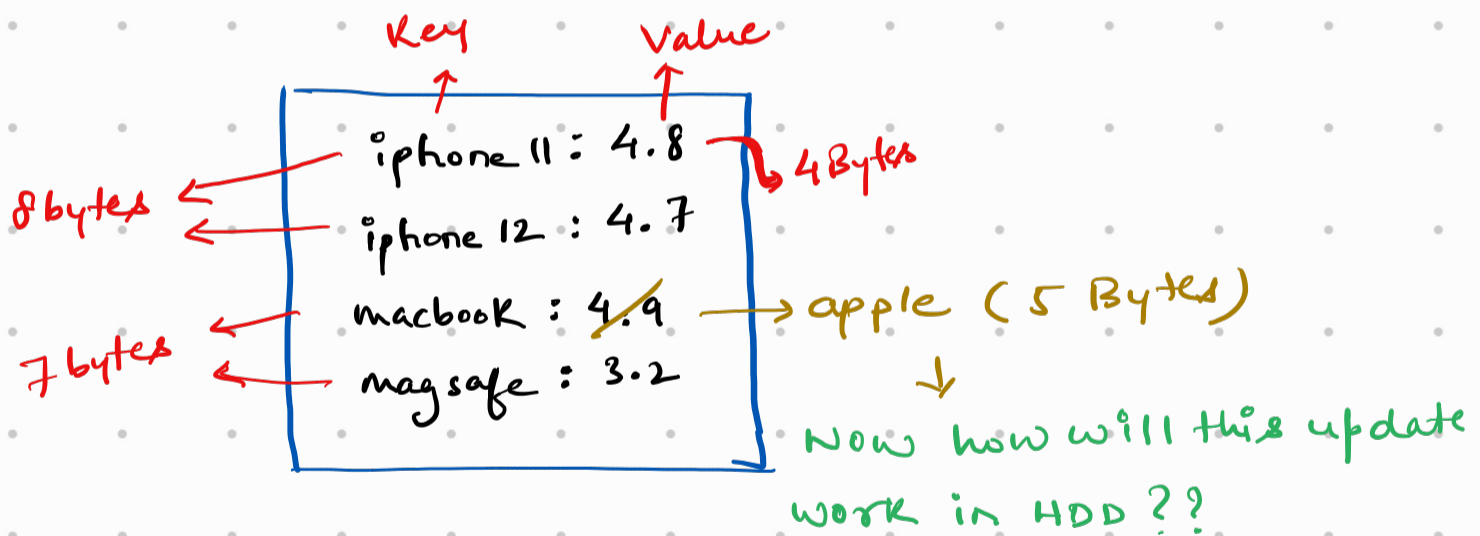
→ xx means some separator value

→ we could have added a separator between each Key-Value pair. Here, we have not.

→ Since the size of Keys and values can be different in Key-value store, so we cannot just apply offset like we used to in RDBMS to reach the next Key.

→ Now, just for a moment, let's assume instead of a decimal num, we updated the value to become a string.

Ex:- `set("macbook") → "apple"`



→ Earlier we had the value 4.9 for macbook which took 4 bytes. Now, the new value "apple" is a string & takes 5 bytes. How will this update work in HDD?

i	p	h	o	n	e	1	1	xx	4
.	8		i	p	h	o	n	e	1
2	xx	4	.	7		m	a	c	b
o	o	k	xx	a	p	p	d	e	a
g	s	a	f	e	xx	3	.	2	

HDD Representation. 1 square = 1 Byte

- First of all, we will find the Key "macbook", then start updating the value.
- This update will end up overriding other data values. In this case, the last character 'e' will override 'u' of the key "magsafe".

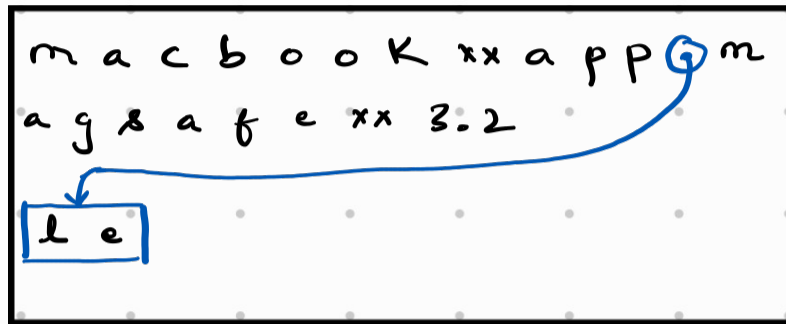
So, how do we solve this?

1) APPROACH 1

What if when we update the data, then we keep writing as we did earlier, but when there is no space left, the :-

- i) Don't override existing data

- i) Split the data and store remaining data somewhere else on the disk.
- ii) Maintain a pointer to the next piece of the splitted data.



- This is a horrific approach w.r.t to the HDD. Because the pointer can lead to anywhere on the HDD, even another sector.
- Jumping across sectors randomly is slow. Hard disks are optimized for sequential access, not random access.

2) APPROACH 2

- Instead of updating, we just insert a new value, like a append-only log file.
- For reads, do linear search from the end.
 - $O(n)$ T.C. (not fast)

How can we optimize this ?

- The idea can come from LRU cache.
- For LRU cache, to find a node fast, we maintain a hashmap of key and address of the node.
- Similarly, we maintain a hashmap of the key and the address on disk where latest data is stored.
- This hashmap we can maintain in our RAM.
- With this, our searches will become $O(1)$.

(Starting Address)

Memory offset



a

iphone11 : 4.9

b

iphone12 : 4.7

c

macbook : 4.8

d

magsafe : 3.2

HDD



Append-only file (like WAL)

Key	Val
iphone11	a
iphone12	b
macbook	c
magsafe	d

In-Memory Hashmap

- To read a Key, first lookup into the hashmap, get the offset & apply that offset on the log file to reach the desired Key in the log file directly instead of sequentially scanning the entire file.
- A property of the WAL file is that we can only append at the last. we can perform insertion or deletion in between.
- Because we cannot delete anything, so our file keeps growing & growing.
- With our current architecture, how will our updates work?
- For updates, we just append in the log file in HDD and update the offset in the hashmap in RAM.

(Starting Address)

Memory offset



a

iphone11 : 4.9

b

iphone12 : 4.7

c

macbook : 4.8

d

magSafe : 3.2

e

macbook : "apple"

HDD



Append-only file (like WAL)

Key	Val
iphone11	a
iphone12	b
macbook	c e
magSafe	d

In-Memory Hashmap

PROBLEMS WITH CURRENT APPROACH

I

Old data is never deleted. So, how do we handle this evergrowing data?

II

For billions of keys (unique), the RAM won't be able to fit the entries. We have to support petabytes of data.

III

Currently we are storing everything into one file. And since we have to store petabytes of data, so, one machine will not be able to fit everything.

IV If system crashes, then data in RAM will be lost.

↳ using the data on HDD, we can reconstruct the offset hashmap on reboot.

LSM Trees

- LSM: Log structured merge trees.
- This will help us resolve all the issues mentioned above.
- Used by :-
 - 1) Cassandra
 - 2) Scylla DB
 - 3) Big Table
 - 4) Couchbase
 - etc.

→ With the current setup of a file (append only) & a hashmap, we can make a few changes and get LSM.

Q Why RDBMS uses WAL?

→ To speed up writes. How? RDBMS writes data on HDD. To improve reads, they support indexes (B^+ trees)

→ At every write on RDBMS, without WAL :

↳ we update data → 1 Random Disk read

↳ Update B^+ tree index → $\log n$ disk read.

↳ & rebalance

→ So instead of updating the index (& Rebalancing) on every write into the disk, RDBMS uses WAL.

In this WAL, we log the recent writes (maybe 100 config batch size) and then when our batch is full, then we move it from WAL to B^+ tree index into the disk.

→ WAL file is also used for master-slave replication.

Q Why NoSQL uses WAL?

NoSQL uses WAL as a part of data storage, mostly facilitated by LSM.

Coming back to our database design

we will use WAL in the following way:-

- 1) we will use WAL to keep appending the data.
- 2) we will maintain a hashmap to keep track of a key's latest offset.

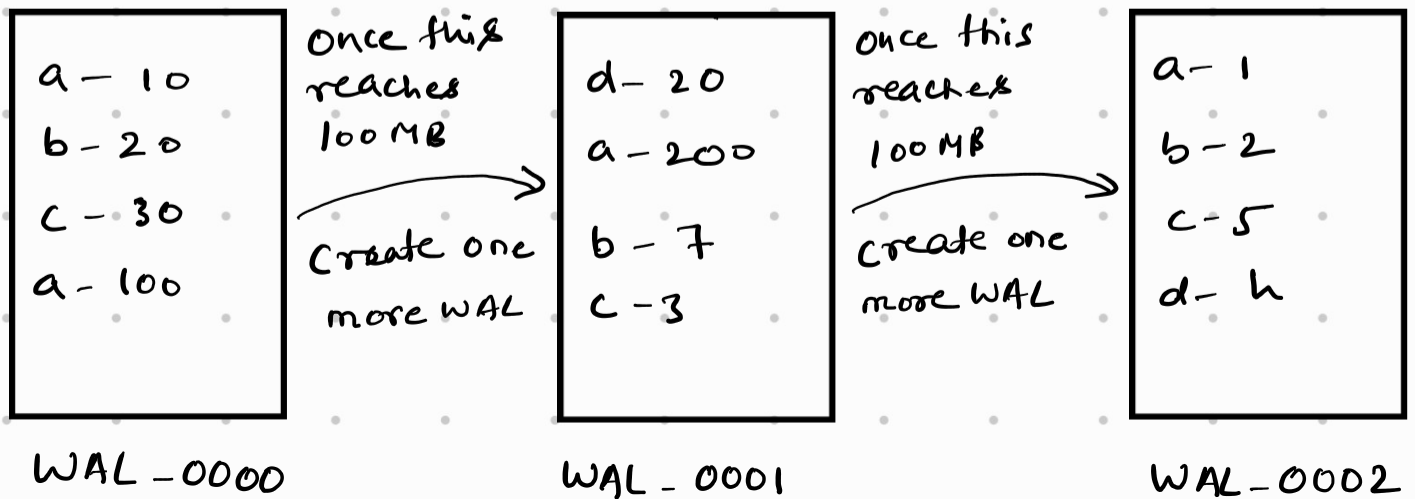
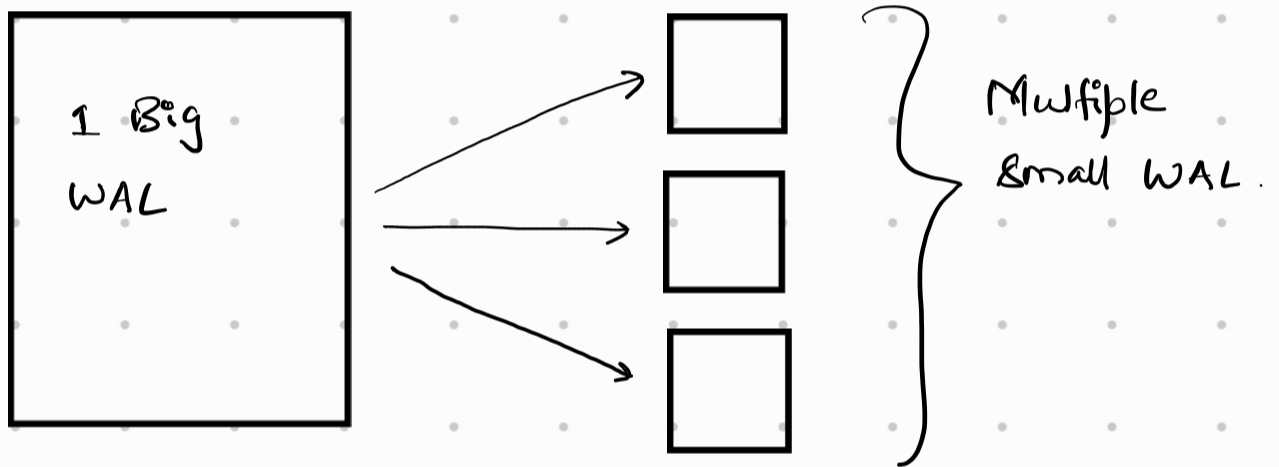
called Compaction

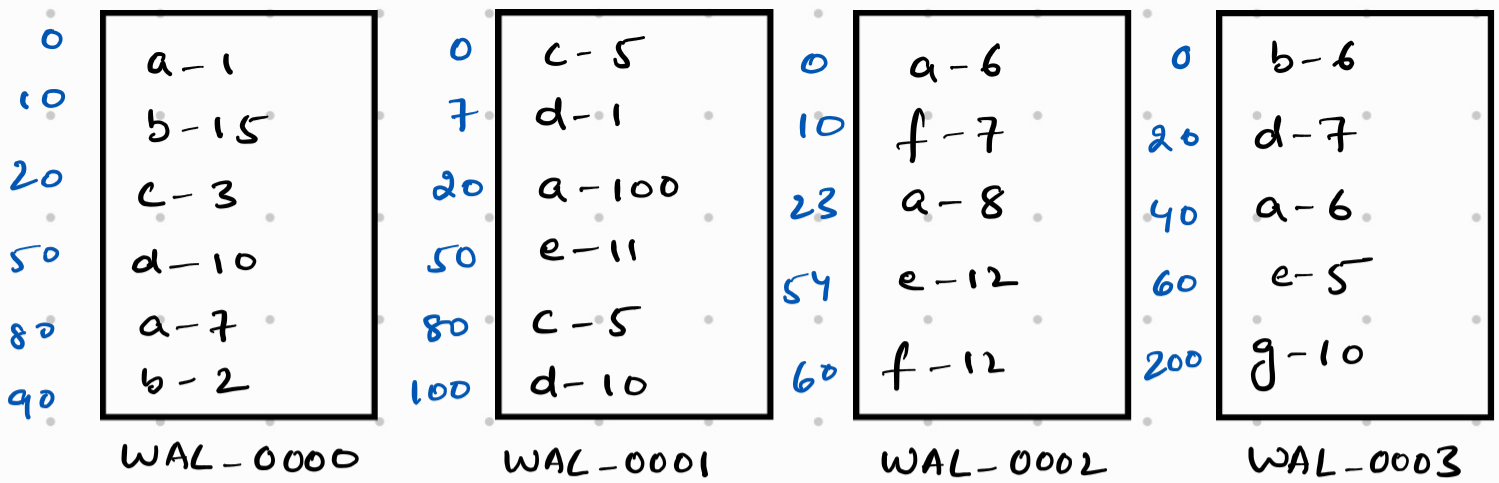
Q How can we remove duplicates from WAL?

- How about we run a periodic background job to remove old entries.
- But with this approach, deletion will be happening in between the file. So, to reclaim the space, will have to shift remaining data.
- Also, when we are performing delete, then we won't be able to take new reads & writes until delete finishes.
- Because our current WAL can be heavy, so this deletion needs to handle the file chunk by chunk, as the complete file is heavy & big.
- Why not maintain multiple smaller size WAL file?

→ Also, in the hashmap, now instead of just storing Key & offset, we can store :-

(Key, WAL file name, offset)





Hash Map

Key	file	offset
a	0003	40
b	0003	0
c	0001	80
d	0003	20
e	0003	60
f	0002	60
g	0003	200

Read $\rightarrow$ get(Key)

$\rightarrow$ Find in hashmap $[O(1)]$ RAM

$\rightarrow$ Find file & offset $[O(1)]$ disk.

Write $\rightarrow$ put(Key)

$\rightarrow$ Append to latest WAL file $[O(1)]$ disk.

$\rightarrow$ Set in hashmap $\rightarrow [O(1)]$ RAM

$\rightarrow$ If WAL file size $> 100\text{MB}$, create a new WAL file.
 $[O(1)]$ disk.

So now, both get & put are efficient.

$\rightarrow$ All writes are happening in active WAL file.

Reads can happen in any WAL file.

$\rightarrow$ Because of this, Compaction process can work on old files.

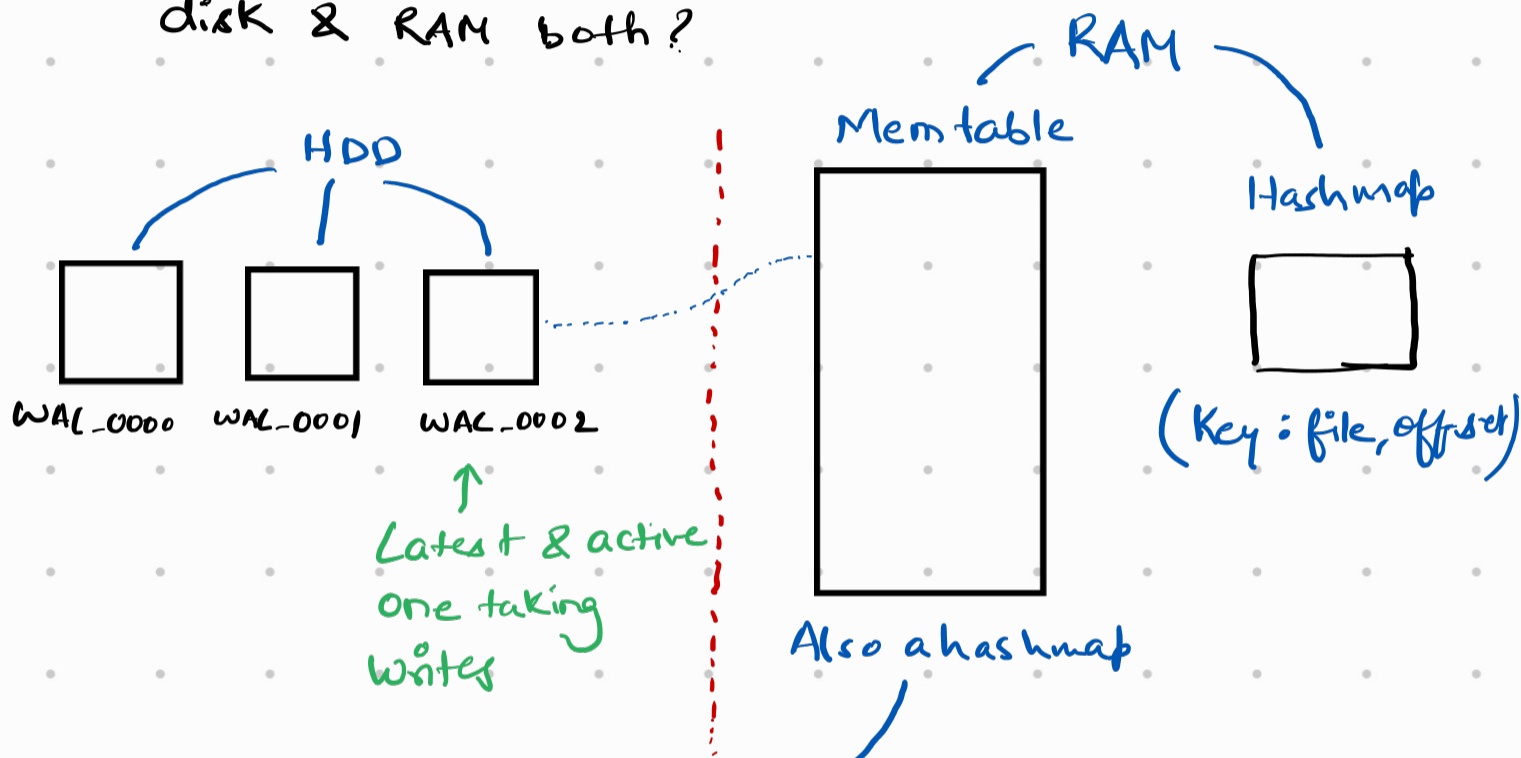
$\rightarrow$ Compaction will not interfere with writes & writes won't be blocked.

$\rightarrow$ Reads might be blocked.

CAN WE IMPROVE FURTHER?

→ Currently, our latest WAL file is in disk only and is of size 100MB only.

→ How about we keep the latest WAL file in disk & RAM both?



→ This hashmap is only going to store whatever new writes we are writing in the latest WAL file.

→ So whatever we have in our latest WAL file, we will have a copy of it inside RAM.

→ we call this hashmap **Memtable**.

So, Memtable refers to the hashmap prepared for the latest WAL file.

→ So, Memtable refers to the RAM version of the active WAL file.

→ why we want it?

→ Disks are not optimized for random access. A hashmap in RAM is optimised for random access.

→ The latest WAL file in disk can contain duplicate keys. The memtable hashmap in RAM contains only unique keys.

→ Also, we can agree that data on memtable is always latest as it's representing active WAL.

→ Now, if we perform read and key is in memtable then read is going to be served from memory, hence making reads fast.

⇒ When we perform write operation, we update memtable, the offset hashmap and also append to WAL file (to ensure durability)

Write = (active WAL + Memtable + offset hashmap)
 Durability Fast reads

⇒ Currently, we have 2 RAM writes + 1 disk write.

FURTHER OPTIMIZATION

⇒ what if while appending to latest WAL file, we don't immediately write to file offset hashmap?

→ This is possible cuz we already have latest data in memtable, we can read from there directly.

→ For older WAL files, we would need the offset hashmap entries.

⇒ So, if we perform a read operation, we will look into the memtable first. If the Key exists there, we will serve from there. If the Key doesn't exist there, that means the Key is part of an old WAL file. So, we will need offset hashmap for that.

So when do we write to offset hashmap?

When our current WAL file becomes full and we are about to create a new one, then using the WAL, we can update the offset hashmap.

⇒ In this way, we have reduced one RAM write.

⇒ So, now we only need to do :-

$\text{write} = (\text{active WAL} + \text{Memtable})$

Q Can we optimize what we do once a WAL file is full?

- 1) So, when WAL (active) is full, we create a new file.
- 2) But what if we dump the active WAL in a new file in a way that we store data in some arranged form?
- 3) So, we can use the memtable (which has all unique value) and take all Key values, sort it in memory, and dump those unique keys in sorted form to a new file.
→ with this, we will be able to achieve some duplicate deletion automatically.
- 4) This file will act as a newly created dump of the last active WAL, but in sorted form with all unique keys.

5) And instead of keeping actual WAL file, we store this new file as it has lesser data (as only unique keys from memtable is added) & sorted data (so searches are faster).

6) This new file is called **Sorted String Table** (**SS Table**).

⇒ In one file we will not have duplicate, but in two different files we can have duplicates.

The following happens once WAL file is full :-

- 1) Flush memtable on disk as a new SSTable.
- 2) SSTable will be sorted by key.
- 3) Update offset hashmap.
- 4) Create a new WAL file and discard old one (as data is in new SSTable in persisted state).

Q Should we clear the memtable?

→ No, we can keep an upper limit to size of memtable.

→ Once we reach the size limit of memtable, we can use some cache eviction policy like LRU.

Q Is the problem of duplicates solved?

- In a single SSTable we don't have duplicates.
- But in 2 different SSTables, duplicates might exist.

Q Is the problem of sharding solved?

- yes. These SSTables doesn't need to be on the same server.
- Also, we can easily replicate it.
- Reads & writes are also fast.
 - As memtable is acting as a cache, and if the hit ratio is good, the data reads will be way faster.

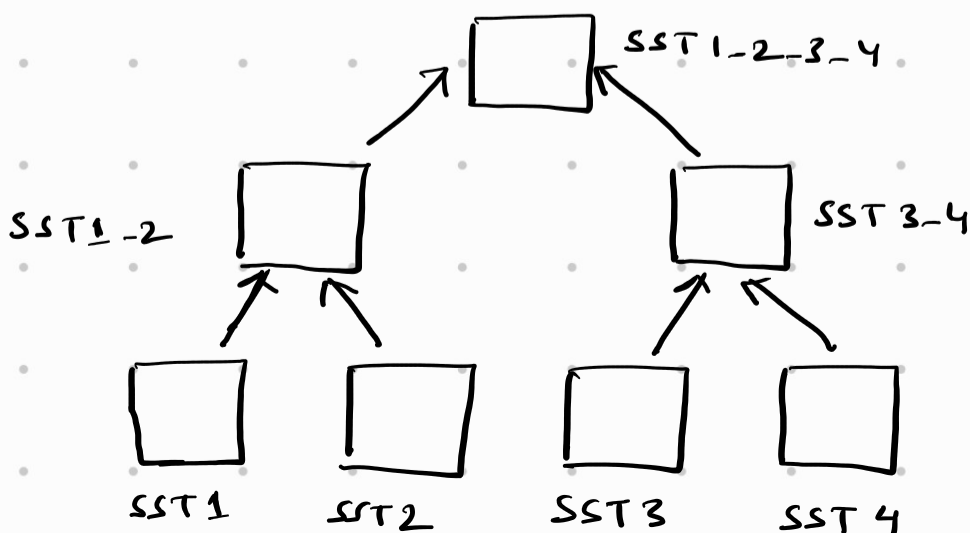
⇒ Now, as majority of the reads are gonna be handled by memtable, and it definitely contains latest data, compaction can happen on WAL files without affecting reads.

↓
SSTable

Compaction

- Operate in background on SSTables.
- SSTables are small, so we can easily load 2-3 tables in RAM.
- Say, we take 2 SSTables, load in RAM & apply "merge 2 sorted array" technique to get a single SSTable with de-duplicated keys.
- To know which SSTable has new value, we can use the file name to determine which value is latest. The file with the higher sequence number is the newer one.
- While merging, we keep latest data & discard the old one.

What if we keep on taking 2 SSTables, merge them & keep doing it recursively??



we keep on compacting till we get a single SSTable.

⇒ Once we have compacted two files, we can remove the old files.

Q Can we eliminate the offset hashmap?

- Currently we use the offset hashmap to find an entry.
- This hashmap can become too large in size.
- If we know which SSTable to search, we can use binary search to get the data.

How do we know which SSTable we should search in?

- Because of compaction, number of SSTable will be less (but of larger size) as we get rid of lower SSTables.

Sparse Index

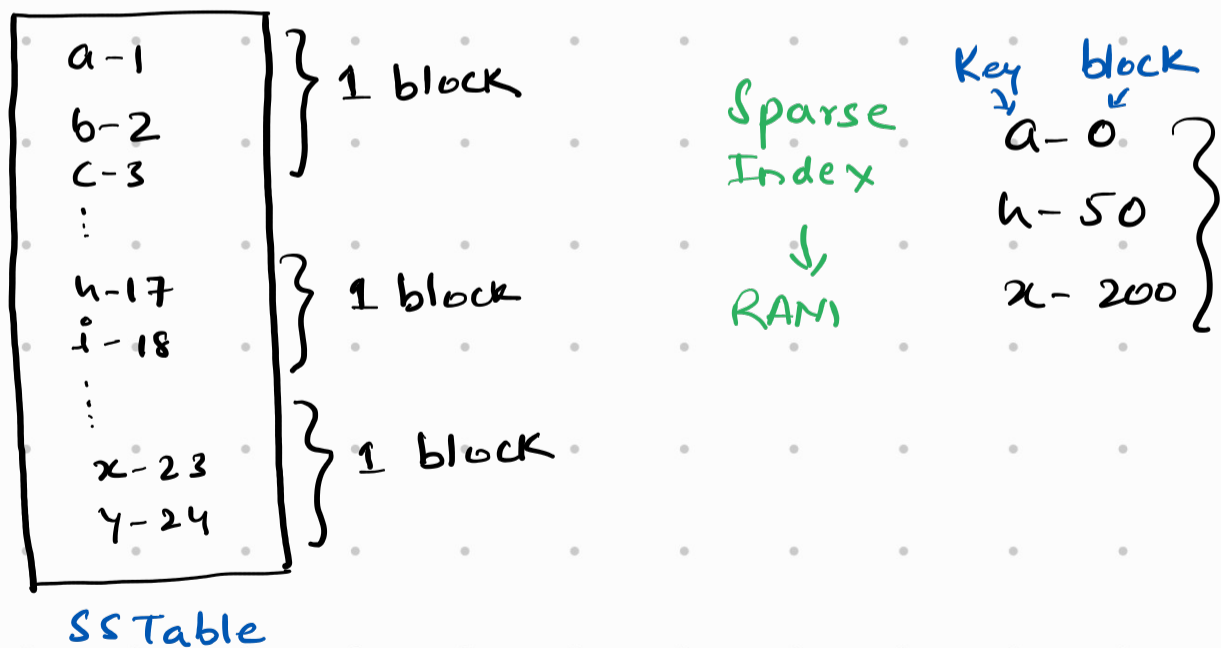
- Till now, we were assuming WAL file size limit to be 100 MB. Now, let's divide our WAL files in small chunks such that each can go on a single block of HDD.

→ Now, if we know the right file, then in one disk read, we get the complete block & we can binary search on it.

WAL → SSTable → Block → Binary Search

So, for each SSTable, we maintain a sparse index

⇒ This sparse index contains starting value of each block.



NOTE → SSTables are immutable, so we can only create it & search on it, no update, delete, insert.

So, how do we delete a Key?

→ We make a new entry of Key with a **sentinel value**. (eg. null)

→ Sentinel value is a special value to mark end of a sequence or data structure.

⇒ Also, to check if a Key is present in LSM or not, we should not go to all SSTable, instead we can use **Bloom filters**.

⇒ For every SSTable, we will have a Bloom filter.

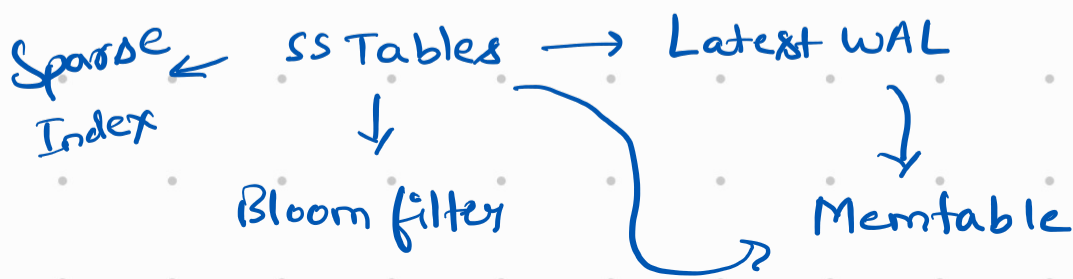
This Bloom filter will let us know about the existence of Key in that SSTable.

⇒ This bloom filter will help us eliminate offset hashmap.

⇒ Also, the space taken by bloom filter will be way less and constant compared to offset hashmap.

⇒ So, if we want to read a Key and its not in Memtable, we can start checking the bloom filters of each SSTable^{starting with the newest one}. If the bloom filter says Key might exist in a particular SSTable, we check the sparse index

of that SSTable, get the block number where that key might exist, read the block and perform binary search. If key is found, then good, otherwise move to the next SSTable (older). And since we keep compacting our SSTables, so by time the no. of SSTables we will need to search in will be less.



Corrections on Few Points

1. "Should we clear memtable?" "yes"

- ⇒ When the active memtable reaches the threshold size, it is frozen and it becomes immutable.
- ⇒ A new fresh memtable is immediately created to continue receiving writes.
- ⇒ A background thread then flushes the immutable memtable to disk as a new SSTable.
- ⇒ Once the flush completes and the SSTable is durable, the immutable memtable is dropped from RAM entirely.
- ⇒ So basically we may have two memtables at a time in RAM for a brief period.
- ⇒ Reads first go to the active memtable, then the immutable one, and then the SSTables starting from the latest one.
- ⇒ As for optimizing reads, we could have a separate cache following LRU. This cache stores frequently accessed keys.

2 Memtable as a plain hashmap that gets sorted at flush time

- ⇒ Actually, memtable maintains sorted order at all times using a concurrent sorted data structure. This provides $O(\log N)$ for insertion.
- ⇒ Once the memtable hits some size threshold, we flush it to disk as a new SSTable. We don't need to sort the data at this point as the keys were already sorted in the memtable. Flushing takes $O(n)$ time.

